Financial Data and Returns

Identify the dates, fields and units of prices before turning them into amounts and returns you can interpret.

Issues	What to understand
1-5: Price data	Query boundaries, OHLC and volume, and starting prices for returns
6-9: Investment amounts	What price returns exclude, stock splits, and buying US stocks with TWD
10-18: Return calculations	Multi-day value, compounding, logs, averages and breaking even
19-21: Checking data	Extreme returns, sharp declines and ticker changes require more than surface checks

The main examples use the same MSFT prices for 2024/8/14-21. The Apple split and Netflix decline use their own actual dates. Share quantities and trades are assumed. Only Issue 6 includes the announced dividend; the other examples use Close for price returns, not total returns including dividends.

ISSUE 01

Which dates do the six downloaded prices cover?

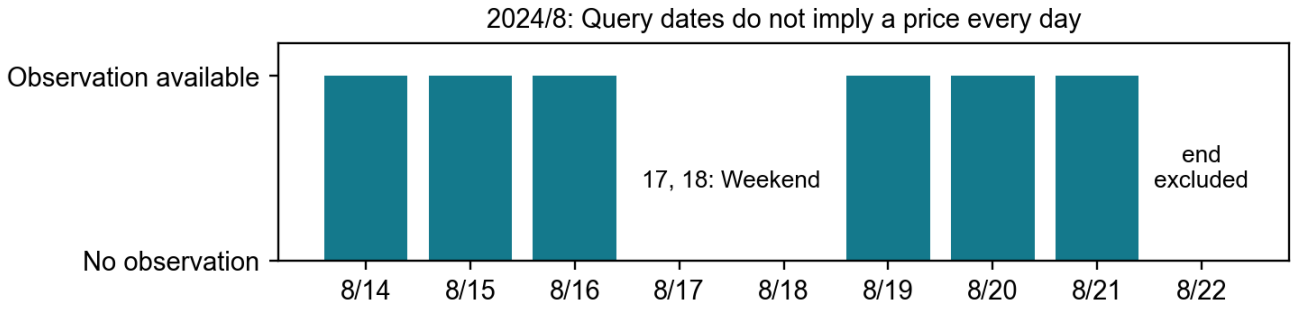
Identify the stock, currency and query boundaries to see where return calculations can begin.

MSFT is Microsoft common stock traded on Nasdaq, priced in USD/share. This query uses daily regular-session data: start includes 8/14, while end excludes 8/22. Do not add prices for weekends.

Before running: Will the output include 8/17, 8/18 or 8/22?

```
import numpy as np
import pandas as pd
import yfinance as yf
msft = yf.Ticker("MSFT").history(
    start="2024-08-14", end="2024-08-22", interval="1d",
    auto_adjust=False, actions=True, prepost=False, keepna=True)
msft.index = msft.index.strftime("%Y-%m-%d")
close = msft["Close"]
pd.DataFrame({"Observation date": close.index,
    "Close (USD/share)": close.map("{:.2f}".format).values})
```

Observation date	Close (USD/share)
2024-08-14	416.86
2024-08-15	421.03
2024-08-16	418.47
2024-08-19	421.53
2024-08-20	424.80
2024-08-21	424.14



Six closing prices form five returns between adjacent trading observations. The observation before 8/19 is 8/16.

Discussion

What is the earliest end date that would also include the 8/22 close?

Sources: [1], [8], [10].

ISSUE 02

What do OHLC prices and volume tell us over six trading days?

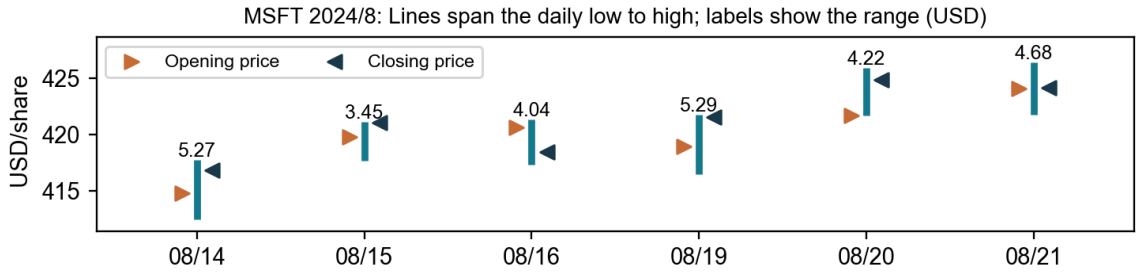
These fields describe market prices and volume, not an investor's complete trades.

Open is the opening price, High/Low the daily high/low, and Close the closing price; all are in USD/share. Volume counts shares traded that day, not people or dollar value. High-low range = daily high-daily low is the daily price range (USD/share).

Before running: Is the 8/15 close of 421.03 the daily high?

```
rows = msft[["Open", "High", "Low", "Close", "Volume"]].copy()
rows["High-low range"] = rows["High"] - rows["Low"]
rows = rows.rename(columns={"Open":"Opening price", "High":"Daily high",
    "Low":"Daily low", "Close":"Closing price", "Volume":"Volume (shares)"})
rows.insert(0, "Observation date", rows.index)
rows.round(2)
```

Observation date	Opening price	Daily high	Daily low	Closing price	Volume (shares)	High-low range
2024-08-14	414.8	417.72	412.45	416.86	18267000	5.27
2024-08-15	419.8	421.11	417.66	421.03	20752100	3.45
2024-08-16	420.6	421.34	417.3	418.47	22775600	4.04
2024-08-19	418.96	421.75	416.46	421.53	15234000	5.29
2024-08-20	421.7	425.86	421.64	424.8	16387600	4.22
2024-08-21	424.08	426.4	421.72	424.14	16067300	4.68



The 8/15 close of 421.03 is below the daily high of 421.11. The close need not be the high or low. Daily data do not show which occurred first; the high-low range is not a guaranteed obtainable profit.

Discussion

Of these six days, volume is highest on 8/16 and lowest on 8/19. What is each high-low range in USD? Does the highest-volume day also have the largest price range here?

Sources: [10], [13].

ISSUE 03

Which five returns can six closing prices provide?

Divide the price change per share by the interval's starting close to measure the change relative to the amount invested.

C is the closing price (USD/share), and r is an interval return in decimal form; C_{t-1} is the previous trading-day close. Six closes form five adjacent returns; 8/16→8/19 remains one interval across the weekend. Assume one share is held, counting only Close price changes, excluding dividends, fees and taxes.

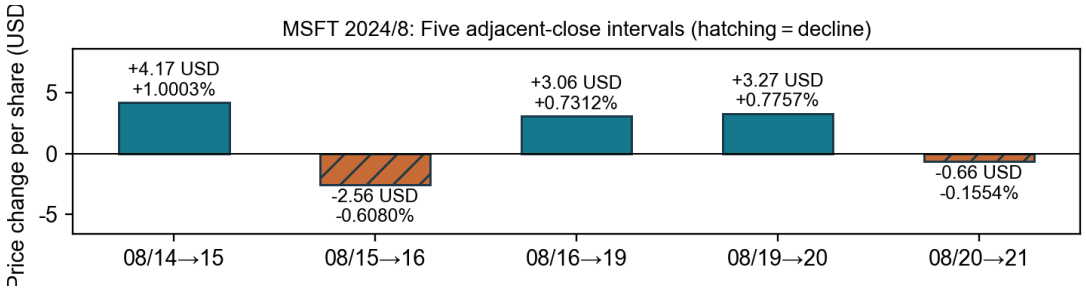
$$r_t = \frac{C_t - C_{t-1}}{C_{t-1}} = \frac{C_t}{C_{t-1}} - 1$$

$$r_{15} = \frac{421.03 - 416.86}{416.86} \approx 0.010003 = 1.0003\%$$

Before running: The price rises by 3.06 USD/share over 8/16→8/19. Is the price return 3.06%?

```
steps = pd.DataFrame({"Start":close.index[:-1], "End":close.index[1:],
    "Starting close":close.values[:-1], "Ending close":close.values[1:]})
steps["Price change per share"] = steps["Ending close"] - steps["Starting close"]
steps["Price return"] = (steps["Ending close"]/steps["Starting close"] - 1).map(
    "{:+.4%}".format)
steps.round(2)
```

Start	End	Starting close	Ending close	Price change per share	Price return
2024-08-14	2024-08-15	416.86	421.03	4.17	+1.0003%
2024-08-15	2024-08-16	421.03	418.47	-2.56	-0.6080%
2024-08-16	2024-08-19	418.47	421.53	3.06	+0.7312%
2024-08-19	2024-08-20	421.53	424.8	3.27	+0.7757%
2024-08-20	2024-08-21	424.8	424.14	-0.66	-0.1554%



Over 8/14→15, the gain is 4.17 USD/share. Dividing by the starting 416.86 gives +1.0003%; +1.0003% is not +1 USD.

Discussion

If you hold 10 shares rather than 1 on 8/14, what are the dollar change and price return by 8/15?

Sources: [7].

ISSUE 04

Why is the 8/14 return blank when its close is present?

A return needs the previous price. The starting observation is missing; an unknown return cannot be treated as 0.

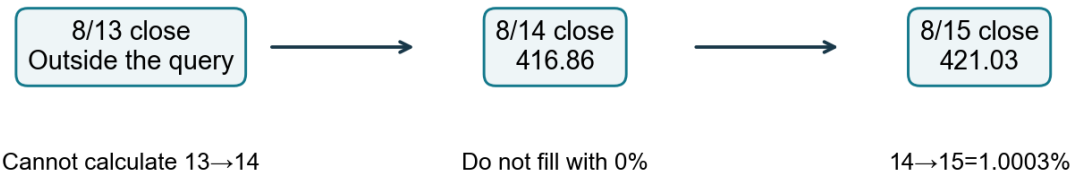
The table in Issue 3 begins with 8/14→15. The query starts on 8/14: its close of 416.86 is present, but the query has no 8/13 close, so 8/13→14 cannot be calculated. The query boundary excludes the previous price; the market price on 8/14 is not missing. pandas returns NaN (not a number) for this row.

$$r_{14} = \frac{C_{14}}{C_{13}} - 1 = \frac{416.86}{?} - 1$$

Before running: The 8/14 close of 416.86 is present. Will the 8/14 return be a number, 0%, or unknown?

```
daily_return = close.pct_change(fill_method=None)
first = pd.DataFrame({"Observation date":close.index[:2],
    "Previous close":close.shift(1).iloc[:2].map(
        lambda x:"Missing" if pd.isna(x) else f"{x:.2f}"),
    "Current close":close.iloc[:2].map("{:.2f}".format),
    "Return":daily_return.iloc[:2].map(
        lambda x:"Unknown" if pd.isna(x) else f"{x:.4%}")})
first
```

Observation date	Previous close	Current close	Return
2024-08-14	Missing	416.86	Unknown
2024-08-15	416.86	421.03	1.0003%



Six prices provide five returns. The 8/14 row also needs the actual 8/13 close.

Discussion

If you replace the unknown 8/14 return with 0%, what 8/13 close in USD are you asserting? Can this query confirm it?

Sources: [1], [7].

ISSUE 05

Can a price rise after opening yet still be down for the day?

Check whether the comparison starts at yesterday's close or today's open.

MSFT: the 8/20 close is 424.80; the 8/21 open is 424.08 and close is 424.14 USD/share. C denotes close and O open; subscripts 20 and 21 are dates in month 8. r_{day} is previous close to close, while $r_{intraday}$ is that day's open to close. Both are price returns with different starting points.

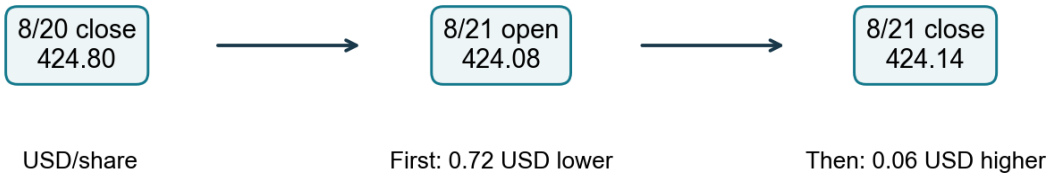
$$r_{\text{day}} = \frac{C_{21}}{C_{20}} - 1, \qquad r_{\text{intraday}} = \frac{C_{21}}{O_{21}} - 1$$

Before running: Does a positive open-to-close return mean today closes above yesterday?

```
c20 = close["2024-08-20"]
o21, c21 = msft.loc["2024-08-21", ["Open", "Close"]]
pd.DataFrame({"Interval":["Previous close to open", "Open to close", "Previous close to close"],
    "Start":[c20,o21,c20], "End":[o21,c21,c21],
    "Return":[f"{o21/c20-1:.4%}", f"{c21/o21-1:.4%}", f"{c21/c20-1:.4%}"]}).round(2)
```

Interval	Start	End	Return
Previous close to open	424.8	424.08	-0.1695%
Open to close	424.08	424.14	0.0142%
Previous close to close	424.8	424.14	-0.1554%

$$r_{\text{day}} = \frac{424.14}{424.80} - 1 \approx -0.1554\%, \qquad r_{\text{intraday}} = \frac{424.14}{424.08} - 1 \approx +0.014\%$$



The price rises 0.06 USD after opening but remains 0.66 USD below the previous close. These statements do not conflict.

Discussion

Investor A buys one share at the 8/21 open (assuming execution at the opening price); investor B already holds one at the 8/20 close. Both still hold at the 8/21 close. What is each investor's profit/loss in USD/share for the day, and which return applies?

Sources: [7].

ISSUE 06

Does the price return include all income from holding a stock?

Calculate price changes separately from dividends for which the holder is eligible. Do not add unannounced future dividends to income in advance.

On 2024/6/12, MSFT announced a dividend of 0.75 USD/share, with an 8/15 ex-dividend date and 9/12 payment date [2]. This known historical event does not imply the same future dividend each period; subsequent dividends still require board declaration [14]. Eligibility here requires buying before the 8/15 ex-dividend date [3]. Assume one share is bought at the 8/14 close and held through 8/15. The close of 421.03 values the stock only; another 0.75 is receivable.

C_{14} and C_{15} are the two closes (USD/share); D is this dividend of 0.75 USD/share, and r is the 14→15 return. The price-only return is about 1.0003%; the calculation below includes the dividend receivable. Ignore taxes, fees, discounting and reinvestment. A receivable is not yet credited cash.

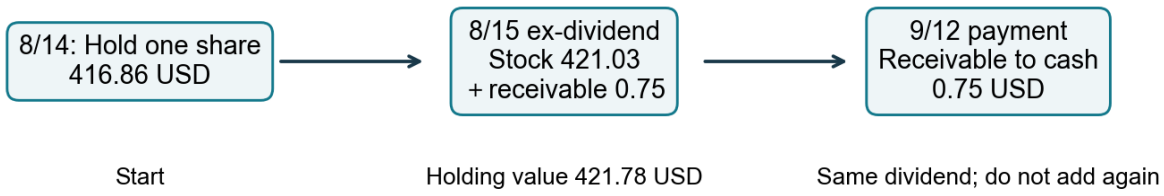
$$C_{15} + D = 421.03 + 0.75 = 421.78 \text{ USD}$$

$$r = \frac{C_{15} + D - C_{14}}{C_{14}} \approx \frac{421.03 + 0.75 - 416.86}{416.86} \approx 1.1803\%$$

Before running: Will adding the dividend receivable raise or lower the price-only return of 1.0003%?

```
c14, c15 = close["2024-08-14"], close["2024-08-15"]
dividend = msft.loc["2024-08-15", "Dividends"]
total15 = c15 + dividend
print(f"8/15 stock {c15:.2f} + dividend receivable {dividend:.2f} = {total15:.2f} USD")
print(f"Price only: {c15/c14-1:.4%}; including the dividend receivable: {total15/c14-1:.4%}")
```

8/15 stock 421.03 + dividend receivable 0.75 = 421.78 USD Price only: 1.0003%; including the dividend receivable: 1.1803%



9/12 is the company's payment date; the actual brokerage credit depends on its processing. Count the same dividend only once. The stock's value on 9/12 requires that day's price and is not estimated here.

Discussion

If the next dividend has not been announced, can you add 0.75 USD/share to the next interval's known income in advance?

Sources: [2], [3], [14].

ISSUE 07

Does a four-for-one split quadruple the value of your holding?

Share quantities and per-share prices must both use pre-split or post-split units. Changing units alone does not increase wealth.

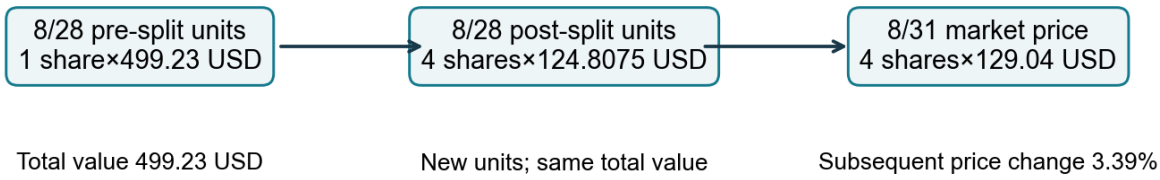
Apple began trading on a four-for-one split basis on 2020/8/31. The Yahoo Close values below already divide pre-split prices by 4 to express them per post-split share: 124.8075 USD on 8/28 corresponds to 499.23 USD for one pre-split share.

$$1 \times 499.23 = 4 \times 124.8075 = 499.23 \text{ USD}$$

Before running: Should the 8/31 return compare 129.04 with 124.8075 or with 499.23?

```
split = yf.Ticker("AAPL").history(start="2020-08-28",
    end="2020-09-01", interval="1d", auto_adjust=False, actions=True)
split.index = split.index.strftime("%Y-%m-%d")
split_close = split["Close"]
pd.DataFrame({"Observation date":split_close.index,
    "Close (USD/share)":split_close.map("{:.4f}".format).values})
```

Observation date	Close (USD/share)
2020-08-28	124.8075
2020-08-31	129.0400



Equivalent unit conversion and the next day's price change are different. Do not divide the already adjusted 124.8075 by 4 again.

Discussion

Can you value 4 post-split shares using the pre-split price of 499.23?

Sources: [1], [4].

ISSUE 08

How much TWD do you need to buy one MSFT share?

Find the required USD amount, then convert using TWD per 1 USD. The units determine whether to multiply or divide.

Assume one share is bought at the 8/15 close of 421.03 USD; first obtain 421.03 USD. X is TWD per 1 USD (TWD/USD). This example uses the Federal Reserve's 8/15 reference value of 32.27 [5]. It is a New York noon reference rate released on 8/19, not an executable bank selling rate on 8/15. Bank quotes and fees change the TWD actually required.

$$421.03\ \text{USD} \times 32.27\ \frac{\text{TWD}}{\text{USD}} \approx 13{,}586.64\ \text{TWD}$$

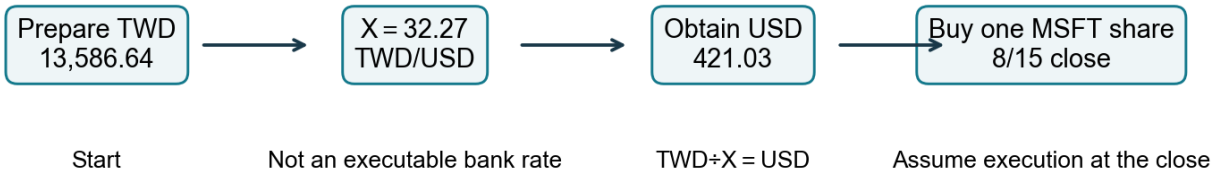
$$13{,}586.64\ \text{TWD} \div 32.27\ \frac{\text{TWD}}{\text{USD}} \approx 421.03\ \text{USD}$$

USD multiplied by TWD/USD cancels USD and gives TWD. TWD divided by TWD/USD gives USD.

Before running: Is the TWD needed for one share 421.03×32.27 or 421.03÷32.27?

```
fx15 = 32.27  # Federal Reserve reference value for 8/15, released on 2024/8/19
twd15 = c15 * fx15  # TWD needed for one share
print(f"Cost of one share: {c15:.2f} USD = {twd15:,.2f} TWD")
print(f"TWD converted back to USD: {twd15/fx15:.2f}; incorrect division: {c15/fx15:.2f}")
```

Cost of one share: 421.03 USD = 13,586.64 TWD TWD converted back to USD: 421.03; incorrect division: 13.05



13,586.64 TWD and 421.03 USD are equivalent at X = 32.27. The units of 421.03÷32.27≈13.05 are neither TWD nor USD; it is not a monetary amount.

Discussion

With only 13,000 TWD, how many USD can you obtain at 32.27, excluding fees? Is it enough for one MSFT share at the 8/15 close?

Sources: [5], [6].

ISSUE 09

If a USD stock price rises, must its TWD value rise?

TWD value depends on both the USD stock price and the exchange rate. Using the same stock prices and changing only the rate isolates its effect in these two cases.

For the same MSFT share, the close rises 421.03→421.53 USD/share over 2024/8/15→19, about 0.1188%. Change only X (TWD/USD): case 1 assumes a constant 32.27; case 2 uses reference values 32.27→32.01. These were released on 8/19 and 8/26 respectively [5][11]. This is retrospective conversion, not a return from actual bank transactions.

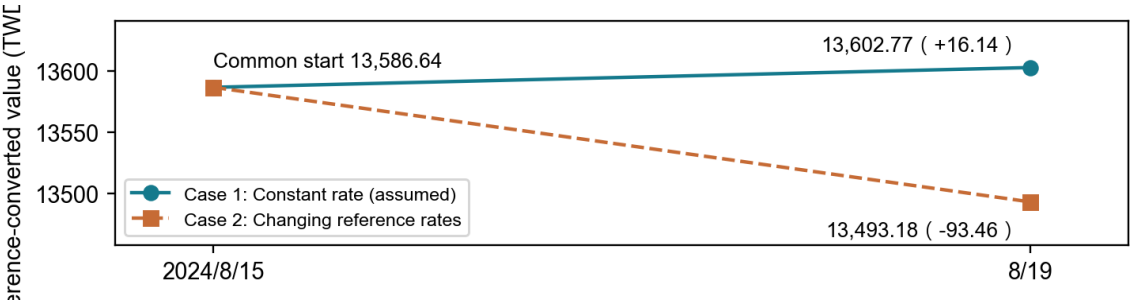
$$(1)\quad \frac{421.53 \times 32.27}{421.03 \times 32.27} - 1 \approx +0.1188\%$$

$$(2)\quad \frac{421.53 \times 32.01}{421.03 \times 32.27} - 1 \approx -0.6879\%$$

Before running: In case 2, will the 8/19 TWD value be above or below 13,586.64?

```
c19, fx19 = close["2024-08-19"], 32.01  # Released on 2024/8/26
fx_cases = pd.DataFrame({"Case":["1 Constant rate (assumed)", "2 Reference rates"],
    "8/15 rate":[fx15, fx15], "8/19 rate":[fx15, fx19]})
fx_cases["8/15 value"] = c15 * fx_cases["8/15 rate"]
fx_cases["8/19 value"] = c19 * fx_cases["8/19 rate"]
fx_cases["TWD return"] = (fx_cases["8/19 value"]/fx_cases["8/15 value"]
    - 1).map("{:+.4%}".format)
fx_cases.round(2)
```

Case	8/15 rate	8/19 rate	8/15 value	8/19 value	TWD return
1 Constant rate (assumed)	32.27	32.27	13586.64	13602.77	+0.1188%
2 Reference rates	32.27	32.01	13586.64	13493.18	-0.6879%



Rates are in TWD/USD and values in TWD/share. With the same stock prices, a constant rate adds 16.14 TWD; the lower reference rate instead reduces value by 93.46 TWD. Parentheses show changes from the common starting value.

Discussion

In case 2, how high must X be on 8/19 for the TWD value to be no lower than 13,586.64 on 8/15?

Sources: [5], [6], [11].

ISSUE 10

Holding one share over six trading days, what is its value after each interval?

Starting market value times r gives the change. Add back the starting value to obtain the updated market value and the next interval's starting amount.

MSFT, 2024/8/14→21: hold one share without trading. W is the stock's market value in USD, not total value including dividends; the 0.75 receivable on 8/15 is separate in Issue 6. r is the interval's decimal price return: 1.0003% = 0.010003. The 1 retains the full starting market value (100%); W×r is only the change, while W×(1+r) includes the starting value.

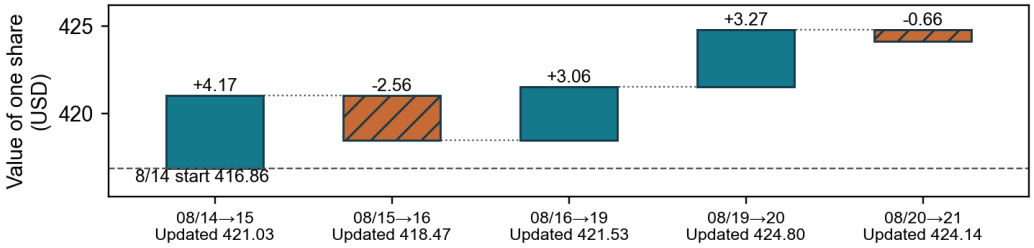
$$W_{\text{end}} = W_{\text{start}}(1 + r) = W_{\text{start}} + W_{\text{start}}r$$

$$416.86(1 + 0.010003) \approx 416.86 + 4.17 = 421.03$$

Before running: Is the starting market value in the 08-16 to 08-19 row 416.86 or 418.47?

```
held = close.iloc[0]  # 8/14 close: one share worth 416.86 USD
path_rows = []
for d0, d1 in zip(close.index[:-1], close.index[1:]):
    seg_r = close[d1]/close[d0] - 1
    path_rows.append([f"{d0[5:]} to {d1[5:]}", held, f"{seg_r:+.4%}",
                      held*seg_r, held*(1+seg_r)])
    held = held*(1+seg_r)  # Becomes the next interval's starting market value
pd.DataFrame(path_rows, columns=["Interval", "Start market value", "Return", "Change", "Updated value"]).round(2)
```

Interval	Start market value	Return	Change	Updated value
08-14 to 08-15	416.86	+1.0003%	4.17	421.03
08-15 to 08-16	421.03	-0.6080%	-2.56	418.47
08-16 to 08-19	418.47	+0.7312%	3.06	421.53
08-19 to 08-20	421.53	+0.7757%	3.27	424.8
08-20 to 08-21	424.8	-0.1554%	-0.66	424.14



Each bar extends from the starting to the updated market value. Its label = starting market value×r (USD). The +3.06 over 8/16→19 is calculated from 418.47, not 416.86.

Discussion

The 8/16 close of 418.47 USD is below 421.03 on 8/15. Is 1+r negative or between 0 and 1 for this interval?

Sources: [7].

ISSUE 11

What is the return from 8/14 to 8/21?

From the second interval onward, each starts at the updated price. Multiply successive growth factors for the total return; do not simply add returns.

MSFT, 2024/8/14→21. r_1 through r_5 are the five returns in order; R is the cumulative price return, and C the close (USD/share). Exclude dividends and fees. For each interval, 1+r = ending price÷starting price; the second interval begins where the first ends. For example, the first two intervals:

$$\frac{421.03}{416.86} \times \frac{418.47}{421.03} = \frac{418.47}{416.86}$$

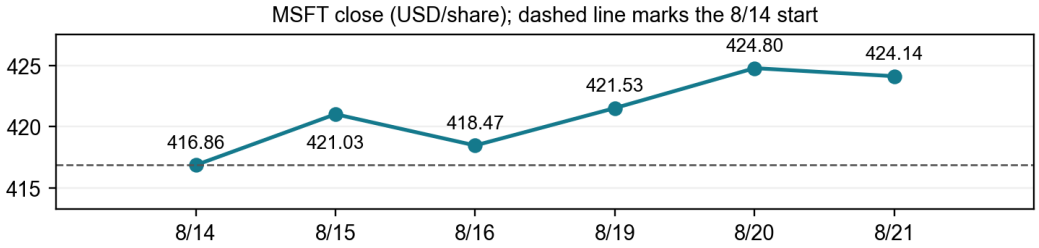
$$1 + R = (1 + r_1)(1 + r_2)\cdots(1 + r_5) = \frac{C_{21}}{C_{14}}$$

$$R = \frac{424.14}{416.86} - 1 \approx 1.7464\%, \qquad r_1 + r_2 + \cdots + r_5 \approx 1.7439\%$$

Before running: Are the final compounded and summed returns equal? Which column equals ending price÷starting price-1?

```
observed_returns = daily_return.dropna()  # Five intervals; no previous close for 8/14
pd.DataFrame({"End":observed_returns.index,
    "Interval return":observed_returns.map("{:+.4%}".format).values,
    "Compounded return":((1+observed_returns).cumprod()-1).map("{:+.4%}".format).values,
    "End÷Start-1":(close[observed_returns.index]/close.iloc[0]-1).map(
        "{:+.4%}".format).values,
    "Sum of returns":observed_returns.cumsum().map("{:+.4%}".format).values})
```

End	Interval return	Compounded return	End÷Start-1	Sum of returns
2024-08-15	+1.0003%	+1.0003%	+1.0003%	+1.0003%
2024-08-16	-0.6080%	+0.3862%	+0.3862%	+0.3923%
2024-08-19	+0.7312%	+1.1203%	+1.1203%	+1.1235%
2024-08-20	+0.7757%	+1.9047%	+1.9047%	+1.8993%
2024-08-21	-0.1554%	+1.7464%	+1.7464%	+1.7439%



The 8/21 price is 7.28 USD above the start, so R = 1.7464%. Simply adding the five returns gives 1.7439%, equivalent to applying each return to 416.86 and ignoring the changing starting value.

Discussion

Knowing only the 8/14 and 8/21 closes, can you calculate the total R? Can you calculate the compounded return in the table's 8/19 row without intermediate prices?

Sources: [7].

ISSUE 12

Why take the log of a price ratio?

Successive ratios are multiplied. Taking natural logs turns multiplication into addition, simplifying accumulation and averaging while allowing recovery of the original ratio.

MSFT closes C on 2024/8/14, 15 and 16: 416.86→421.03→418.47 USD/share. ln is the natural logarithm; exp(x) = e raised to x (e≈2.71828), and exp(ln a) = a. ℓ is an interval's decimal log return. ln turns multiplication of positive ratios into addition; it does not make the original returns more accurate. Prices and ratios must be greater than 0.

$$\ell_t = \ln \frac{C_t}{C_{t-1}}; \qquad \ln(ab) = \ln a + \ln b \quad (a, b > 0)$$

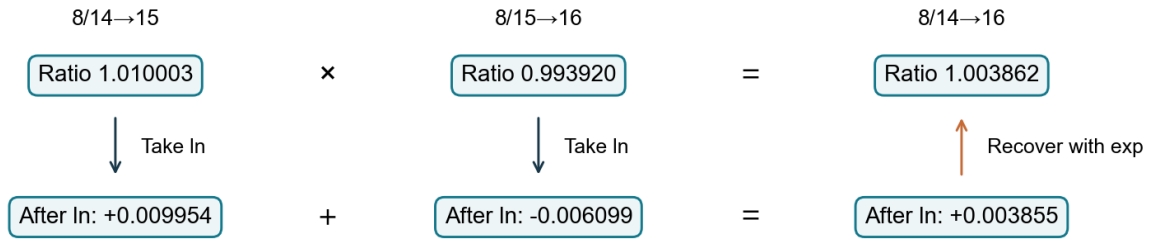
a and b are the two positive price ratios: a = 421.03÷416.86≈1.010003 and b = 418.47÷421.03≈0.993920. ln is positive for a ratio above 1, 0 at 1, and negative between 0 and 1. The formula shows six-decimal approximations; the code retains full precision.

$$\ln(1.010003 \times 0.993920) \approx 0.009954 - 0.006099 = 0.003855$$

Before running: Will the sum of the two log returns equal ln(418.47÷416.86)?

```
c16 = close["2024-08-16"]  # Third close in the same data
ratio15, ratio16 = c15/c14, c16/c15
log15, log16 = np.log(ratio15), np.log(ratio16)
print(f"Product of ratios: {ratio15:.6f}×{ratio16:.6f} = {ratio15*ratio16:.6f}")
print(f"Sum of ln: {log15:+.6f} + {log16:+.6f} = {log15+log16:+.6f}")
print(f"ln(418.47÷416.86) = {np.log(c16/c14):+.6f}; recovered with exp: {np.exp(log15+log16):.6f}")
```

Product of ratios: 1.010003×0.993920 = 1.003862 Sum of ln: +0.009954 + -0.006099 = +0.003855
ln(418.47÷416.86) = +0.003855; recovered with exp: 1.003862



Top: the product of the two price ratios = 1.003862 = 418.47÷416.86. Bottom: the sum after taking ln = +0.003855. exp(0.003855) recovers 1.003862.

Discussion

The 8/16→8/19 price ratio is 421.53÷418.47≈1.007312. Without a calculator, is its ln positive, negative or 0? If a close is unchanged over an interval, what are its ratio and ln?

Sources: [7].

ISSUE 13

Does a smaller log return mean less money was earned from the same two prices?

Simple and log returns express the same price ratio in different ways. Their numbers differ, but dollar profit/loss is the same.

MSFT closes rise from 416.86 on 2024/8/14 to 421.03 USD/share on 8/15. Multiply the starting amount by simple return r for profit/loss; log return $\ell = \ln(1+r)$ is convenient to add. ℓ is about 0.0099537, also displayed as 0.9954%; exp requires the decimal value, not 0.9954. Retain more digits when recovering the return to avoid rounding the last digit. The numerical difference is neither a fee nor lost earnings.

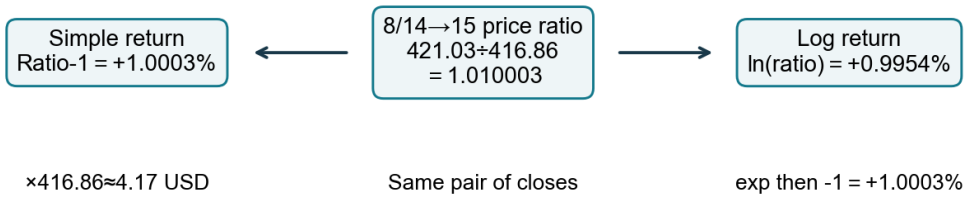
$$r = \frac{421.03}{416.86} - 1 \approx 1.0003\%; \quad \ell = \ln\frac{421.03}{416.86} \approx 0.0099537$$

$$r = \exp(\ell) - 1; \qquad \exp(0.0099537) - 1 \approx 1.0003\%$$

Before running: Will converting back from the log return recover 1.0003% or a different gain?

```
r15 = c15/c14 - 1
recovered15 = np.expm1(log15)  # expm1(x)=exp(x)-1
print(f"Simple return: {r15:.4%}; log return: {log15:.4%}")
print(f"Recovered from log: {recovered15:.4%}; ×{c14:.2f} = P/L per share: {c14*recovered15:.2f} USD")
```

Simple return: 1.0003%; log return: 0.9954% Recovered from log: 1.0003%; ×416.86 = P/L per share: 4.17 USD



Incorrectly treating 0.9954% as a simple return and multiplying by 416.86 gives about 4.15 USD, below the actual 4.17. The calculation is misused; no earnings are lost.

Discussion

Over 8/15→8/16, the simple return is about -0.6080% and log return about -0.6099%. Does the more negative log return mean a one-share holder lost more money?

Sources: [7].

ISSUE 14

How do five summed log returns recover the total price gain?

Log returns can be added directly. Apply exp afterward to recover the actual simple return and price.

MSFT, five intervals of positive prices over 2024/8/14→21. ℓ_i is interval i's log return; i = 1 is 14→15 and i = 5 is 20→21. Σ means summing the five intervals. The result remains on the log scale. R is the total simple price return, excluding dividends and fees. C is the close in USD/share.

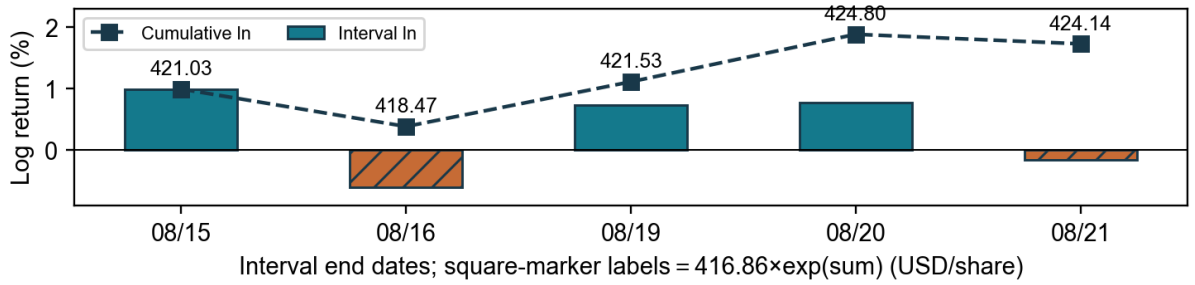
$$\sum_{i=1}^{5} \ell_i = \ln \frac{C_{21}}{C_{14}} \approx \ln \frac{424.14}{416.86} \approx 0.017313$$

$$R = \exp\left(\sum \ell_i\right) - 1 \approx 1.7464\%; \quad C_{21} = C_{14}(1+R)$$

Before running: The five log returns sum to about 0.017313. Can this be called a price return of 1.7313% directly?

```
log_returns = np.log(close/close.shift(1)).dropna()
log_sum = log_returns.sum()
recovered_total = np.expm1(log_sum)
print(f"Sum of five ln returns: {log_sum:.6f}; ln(424.14÷416.86): {np.log(close.iloc[-1]/close.iloc[0]):.6f}")
print(f"Recovered R: {recovered_total:.4%}; recovered 8/21 close: {close.iloc[0]*np.exp(log_sum):.2f} USD")
```

Sum of five ln returns: 0.017313; ln(424.14÷416.86): 0.017313 Recovered R: 1.7464%; recovered 8/21 close: 424.14 USD



The sum through 8/21 is 0.017313; applying exp then -1 gives R = 1.7464%. 416.86×R gives profit/loss of 7.28 USD, while 416.86×(1 + R) recovers 424.14. Square markers show the recovered closes.

Discussion

If you treat the summed log return of 1.7313% as a simple return, what 8/21 close follows from 416.86? How far is it from the actual 424.14?

Sources: [7].

ISSUE 15

What is the arithmetic mean of five returns, and what does it tell us?

Sum the five observed price returns and divide by 5. This is a historical average, not a gain earned every day.

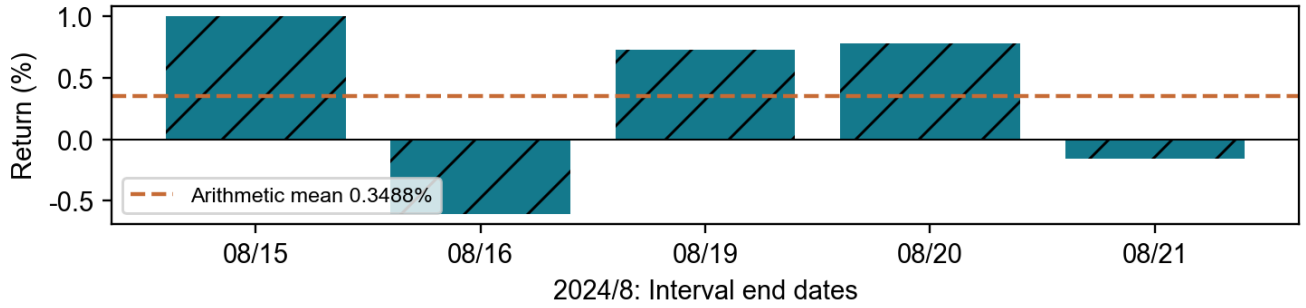
The intervals are 8/14→15, 15→16, 16→19, 19→20 and 20→21, using Close price returns without dividends or fees. n = 5, and the bar above r denotes the arithmetic mean. It measures the average change between adjacent closes over these five intervals. Use it to compare interval-return levels for the same stock over different periods with equal interval counts. It neither guarantees the next return nor preserves the order of gains and losses.

$$\bar{r} = \frac{1}{n}\sum_{i=1}^{n} r_i \approx \frac{1.0003 - 0.6080 + 0.7312 + 0.7757 - 0.1554}{5}\% \approx 0.3488\%$$

Before running: How many of the five returns exceed the mean of about 0.3488%?

```
arithmetic = observed_returns.mean()
print(f"Sum of five returns: {observed_returns.sum():.4%}; interval count: {observed_returns.count()}")
print(f"Arithmetic mean: {arithmetic:.4%}; intervals above the mean: {(observed_returns > arithmetic).sum()}")
```

Sum of five returns: 1.7439%; interval count: 5 Arithmetic mean: 0.3488%; intervals above the mean: 3



The mean is 0.3488%; three returns are above it and two below, with none exactly equal. The mean does not retain their order or predict the next return.

Discussion

8/16→8/19 crosses the weekend. What mean results if you split it into three intervals and insert 0% for the two weekend intervals? Why should you not do this?

Sources: [7], [8].

ISSUE 16

What questions do arithmetic and geometric means answer?

One averages observed interval returns; the other converts the compounded total into an equivalent per-interval growth rate.

MSFT, five price returns r over 2024/8/14→21, excluding dividends and fees. The arithmetic mean is the sum of five r values÷5. The geometric mean g reaches the same endpoint by multiplying (1+g) over all five intervals. The exponent 1/5 takes a fifth root, not a fifth power. Decimal g is an equivalent calculation, not an observed daily return. Keep frequency fixed when comparing periods of different lengths. All 1+r are positive and the five returns differ, so the arithmetic mean exceeds the geometric mean here.

$$\bar{r} = \frac{r_1 + \cdots + r_5}{5} \approx 0.3488\%$$

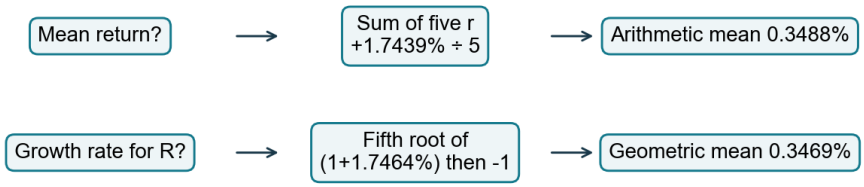
$$g = [(1 + r_1)\cdots(1 + r_5)]^{1/5} - 1 \approx \left(\frac{424.14}{416.86}\right)^{1/5} - 1 \approx 0.3469\%$$

ln converts the product of five ratios into a sum. Divide that sum by 5 and apply exp to obtain the total ratio's fifth root. Thus exp(mean of the five log returns)-1 also gives g.

Before running: Will the geometric mean be above or below the arithmetic mean of 0.3488%? Which mean will the third line equal?

```
geometric = (1 + observed_returns).prod()**(1/5) - 1
print(f"Arithmetic mean: {arithmetic:.4%} (sum of five r values÷5)")
print(f"Geometric mean: {geometric:.4%} (multiply, take the fifth root, then subtract 1)")
print(f"exp(mean of five ln returns)-1: {np.expm1(log_returns.mean()):.4%}")
```

Arithmetic mean: 0.3488% (sum of five r values÷5) Geometric mean: 0.3469% (multiply, take the fifth root, then subtract 1) exp(mean of five ln returns)-1: 0.3469%



The arithmetic mean of 0.3488% averages the five r values. The geometric mean of 0.3469% converts the total 1.7464% into equivalent per-interval growth; Issue 17 uses it to reproduce the endpoint.

Discussion

To reproduce the endpoint, repeat the calculated g five times (not market observations). What are the arithmetic and geometric means of those five values? Why do the means differ for the original five returns?

Sources: [7].

ISSUE 17

Which mean, repeated five times, reproduces the 8/21 close?

Repeating the geometric mean over five intervals reaches the actual endpoint. This constant-growth path is a calculation tool, not market prices.

MSFT: closing C = 416.86 on 2024/8/14 and 424.14 USD/share on 8/21. The five intervals count only price changes. g is about 0.3469%, and the arithmetic mean about 0.3488%. Repeating g reaches the same endpoint, but intermediate prices are neither forecasts nor market observations. The formula uses approximate rates; the code uses full precision.

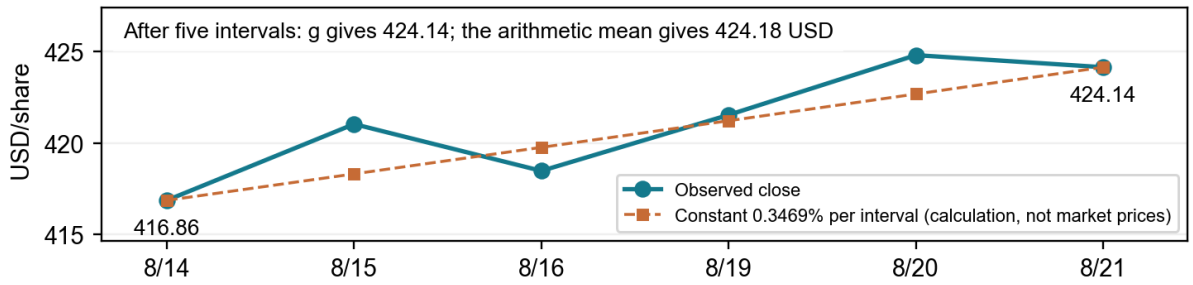
$$(1+g)^5 = \frac{C_{21}}{C_{14}}; \qquad 416.86(1+0.003469)^5 \approx 424.14$$

$$416.86(1+0.003488)^5 \approx 424.18$$

Before running: Will repeating the arithmetic mean five times give exactly 424.14 USD?

```
c21 = close["2024-08-21"]
print(f"Geometric mean repeated 5 times: {c14*(1+geometric)**5:.2f} USD; actual 8/21: {c21:.2f}")
print(f"Arithmetic mean repeated 5 times: {c14*(1+arithmetic)**5:.2f} USD")
```

Geometric mean repeated 5 times: 424.14 USD; actual 8/21: 424.14 Arithmetic mean repeated 5 times: 424.18 USD



Across the six observation dates, the two paths match only at the start and end. Connecting lines aid reading; they are not additional price observations. Repeating the arithmetic mean five times gives 424.18, which exceeds 424.14 by 0.04 USD.

Discussion

Someone interprets the geometric mean of 0.3469% as “MSFT rose 0.3469% every day in these five intervals.” If true, what would the 8/16 close be? Compare it with the actual 8/16 close of 418.47 to explain the error.

Sources: [7].

ISSUE 18

Why can a positive average return still fail to break even?

A rise after a decline starts from a lower price. Even a larger percentage gain may not recover the original amount.

MSFT closes on 2020/3/6, 3/9 and 3/10 are 161.57, 150.62 and 160.92 USD/share, forming two price-return intervals: a decline followed by a rise.

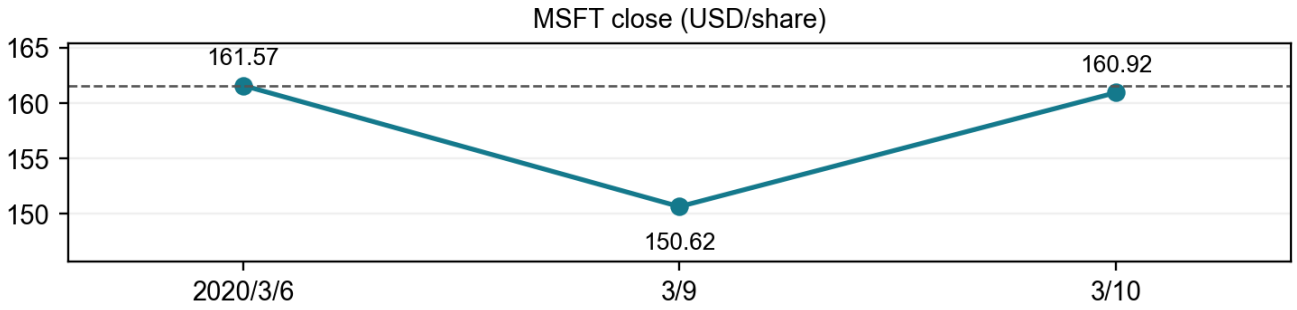
Before running: The second gain of 6.8384% exceeds the earlier decline of 6.7773%. Does this guarantee breaking even?

```
march = yf.Ticker("MSFT").history(start="2020-03-06",
    end="2020-03-11", interval="1d", auto_adjust=False, actions=True)
march.index = march.index.strftime("%Y-%m-%d")
mc = march["Close"]
mr = mc.pct_change(fill_method=None).dropna()
print(f"Two interval returns: {mr.iloc[0]:+.4%}, {mr.iloc[1]:+.4%}")
print(f"Arithmetic mean: {mr.mean():+.4%}; total return: {mc.iloc[-1]/mc.iloc[0]-1:+.4%}")
```

Two interval returns: -6.7773%, +6.8384% Arithmetic mean: +0.0306%; total return: -0.4023%

The arithmetic mean is +0.0306%, but the 3/10 price remains 0.65 USD below 3/6. Recovering from 150.62 on 3/9 to 161.57 on 3/6 requires:

$$\frac{161.57}{150.62} - 1 \approx 7.2700\%$$



The actual rebound of 6.8384% is below the 7.2700% needed to break even. This is an observed counterexample, not an estimate of its frequency.

Discussion

The 3/10 close has rebounded to 160.92. Why has it still not broken even?

Sources: [7], [10].

ISSUE 19

What should you check first after calculating a -74% daily return?

A large return may reflect inconsistent units, a corporate event or a real price change. Investigate before deciding whether to use it.

Apple, 2020/8/28 and 31. Both snapshot Close values already use post-split units: 124.8075 and 129.04 USD/share. Converting the 28th back to old-share units gives 4×124.8075 = 499.23. Dividing the 31st's new-share price of 129.04 by the 28th's old-share price of 499.23 deliberately demonstrates an inconsistent-unit calculation, not an error in the original snapshot prices. The company announcement confirms the four-for-one split [4], but does not independently verify each price.

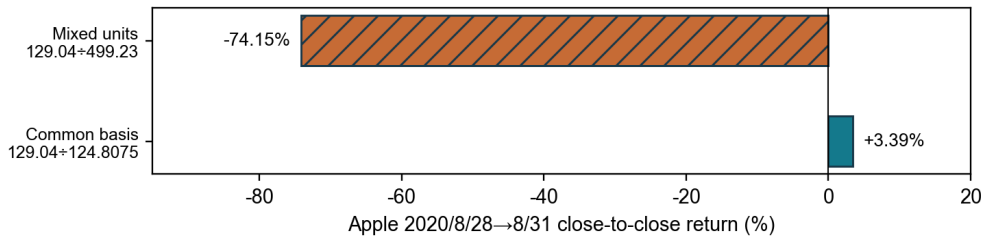
$$\frac{129.04}{4 \times 124.8075} - 1 = \frac{129.04}{499.23} - 1 \approx -74.15\%, \qquad \frac{129.04}{124.8075} - 1 \approx +3.39\%$$

Price pairing	8/28 start	8/31 end	Calculated result
Mixed old/new share units	499.23 (old share)	129.04 (new share)	About -74.15%
Both in new-share units	124.8075 (new share)	129.04 (new share)	About +3.39%

Before running: Will Stock Splits on 8/31 be 0 or 4? What approximate return results from mixing units?

```
split_ratio = split.loc["2020-08-31", "Stock Splits"]
mixed = split_close.iloc[1] / (4*split_close.iloc[0]) - 1
same_basis = split_close.iloc[1] / split_close.iloc[0] - 1
print(f"8/31 Stock Splits : {split_ratio:.0f}")
print(f"Mixed units: {mixed:+.2%}; same post-split basis: {same_basis:+.2%}")
```

```
8/31 Stock Splits : 4 Mixed units: -74.15%; same post-split basis: +3.39%
```



The mixed-unit calculation divides a post-split share price of 129.04 by a pre-split share price of 499.23. On a common basis, the 8/31 price change is +3.39%, not a shareholder loss of 74%.

Discussion

Given the four-for-one split on 8/31, how much are the four shares on 8/31 worth in USD if compared in pre-split units? Is the return the same as on a common post-split basis?

Sources: [1], [4].

ISSUE 20

Can Netflix’s 35% daily decline be deleted under a -20% rule?

A large decline does not imply bad data. Deleting a real decline solely for its size hides this loss from the analysis.

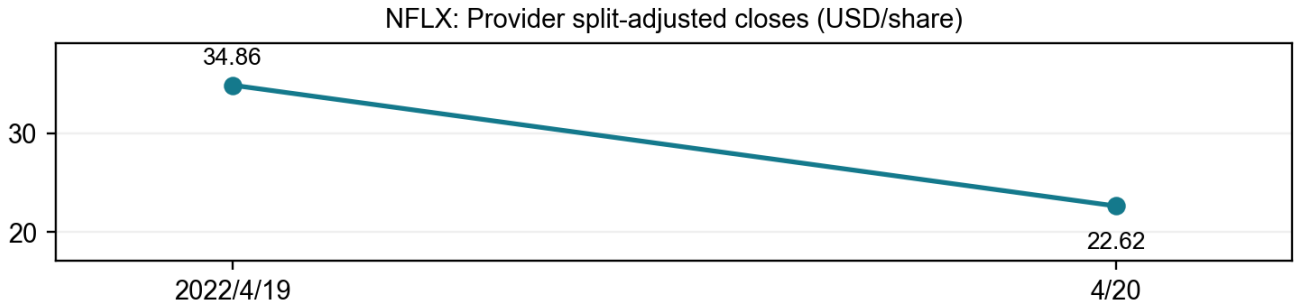
The false decline in Issue 19 comes from mixed units. For NFLX over 2022/4/19→20, both Close values use the provider’s common post-split basis, not the original quotes in 2022. Stock Splits is 0 on both days; share units are consistent below. Assume the rule “delete the day’s price if the return is below -20%.” Crossing this threshold does not prove a price error. The shareholder letter [9] provides event context, not a substitute for checking prices or proof of a single cause of the decline.

Before running: Will this observed event remain in the sample if deletion uses only the -20% threshold?

```
nflx = yf.Ticker("NFLX").history(start="2022-04-19",
    end="2022-04-21", interval="1d", auto_adjust=False, actions=True)
nflx.index = nflx.index.strftime("%Y-%m-%d")
nc = nflx["Close"]
crash = nc.iloc[1]/nc.iloc[0]-1
print(f"4/19：{nc.iloc[0]:.3f}；4/20：{nc.iloc[1]:.3f} USD/share")
print(f"Sum of Stock Splits over both days: {nflx['Stock Splits'].sum():.0f}")
print(f"Price return: {crash:.2%}; deletion threshold triggered: {'Yes' if crash < -.20 else 'No'}")
```

4/19：34.861；4/20：22.619 USD/share Sum of Stock Splits over both days: 0 Price return: -35.12%; deletion threshold triggered: Yes

$$r = \frac{22.619}{34.861} - 1 \approx -35.12\%$$



The -20% rule deletes -35.12%. This threshold has not shown that either 34.861 or 22.619 is incorrect.

Discussion

After deleting the 4/20 price, can the remaining single observation still provide the 19→20 decline?

Sources: [1], [9].

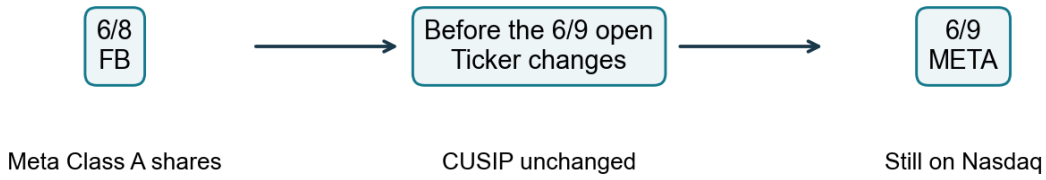
ISSUE 21

Does changing FB to META create a different security?

A ticker identifies a security; it is not a price. Confirm that it remains the same security before connecting historical prices.

Meta announced that before the 2022/6/9 open, its Nasdaq Class A common stock ticker would change from FB to META while its CUSIP security identifier remained unchanged. This supports continuity of the same security, not merely the same company.

Observation date (2022)	Ticker	Share class	Market
6/8	FB	Class A common stock	Nasdaq
6/9	META	Class A common stock	Nasdaq



A failed FB query does not mean historical prices did not exist; a provider may supply them under the current META ticker.

The announcement does not establish that two price files can be merged directly. Still check duplicate dates, Close versus Adj Close, currency and split basis. A ticker change alone does not create a new investment.

Discussion

Which announcement detail supports continuity of the same security more directly than having the same company name?

Sources: [12].

References and Reproduction

Prices, corporate events and exchange rates each have their own sources.

[1] yfinance: Historical price parameters

[2] Microsoft: 2024/6/12 dividend announcement

[3] Investor.gov: Ex-dividend dates and dividend eligibility

[4] Apple: 2020 four-for-one split announcement

[5] Federal Reserve: 2024/8/19 exchange-rate release

[6] FRED: TWD/USD exchange rates and observation timing

[7] Zivot: Return calculations

[8] Nasdaq: Trading hours and calendar

[9] Netflix: 2022 quarter 1 shareholder letter

[10] Yahoo Finance: MSFT historical prices

[11] Federal Reserve: 2024/8/26 exchange-rate release

[12] Meta: SEC announcement changing FB to META

[13] Nasdaq: Volume

[14] Microsoft: 2025 annual report; dividends still require board declaration

Use the retained Yahoo/yfinance observations, without replacing them with new downloads. The main MSFT data cover 2024/8/14-21; split and decline examples state their own dates. Exchange rates are published reference values, not prices executed simultaneously with stock closes.

Run each issue’s code in order; its final table expression displays in a notebook. numpy, pandas and yfinance are required. Formula substitutions use displayed precision; code retains full precision. Fees, taxes and unspecified dividends are excluded. A short sample is not an estimate of future returns.